

AUTOMATED DEDUCTION IN QUASIGROUP & LOOP THEORY

Michael Kinyon



AITP 2026
29 August 2026

Automated vs Interactive Theorem Provers

What has changed now that Fields Medalists have made formalization mainstream? Now when I tell mathematicians I use theorem-proving software, they almost always assume I am talking about Lean. (Yes, I am serious: *almost always*.)

In fact, I am not referring to formalization nor to any *interactive theorem prover* (ITP). I am talking about *automated theorem provers* (ATPs).

Nowadays, the primary use case of ATPs is by ITPs to handle various tasks, e.g., hammers.

I am more interested in direct use of ATPs to discover new mathematics and settle conjectures.

Why Prover9?

There are several good ATP systems: E, Vampire, Twee, iProver, etc. So why do mathematicians who use ATPs, still prefer PROVER9, even though until recently, it was unsupported legacy software?

Primary reasons:

Why Prover9?

There are several good ATP systems: E, Vampire, Twee, iProver, etc. So why do mathematicians who use ATPs, still prefer PROVER9, even though until recently, it was unsupported legacy software?

Primary reasons:

- Prover9 had an OK GUI.

Why Prover9?

There are several good ATP systems: E, Vampire, Twee, iProver, etc. So why do mathematicians who use ATPs, still prefer PROVER9, even though until recently, it was unsupported legacy software?

Primary reasons:

- Prover9 had an OK GUI. (Remember: most software users would rather have painful dental surgery than use a command line.)

Why Prover9?

There are several good ATP systems: E, Vampire, Twee, iProver, etc. So why do mathematicians who use ATPs, still prefer PROVER9, even though until recently, it was unsupported legacy software?

Primary reasons:

- Prover9 had an OK GUI. (Remember: most software users would rather have painful dental surgery than use a command line.)
- Prover9's input/output language is easily read out of the box. Most other systems rely on TPTP, which is hard to use for beginners.

Why Prover9?

There are several good ATP systems: E, Vampire, Twee, iProver, etc. So why do mathematicians who use ATPs, still prefer PROVER9, even though until recently, it was unsupported legacy software?

Primary reasons:

- Prover9 had an OK GUI. (Remember: most software users would rather have painful dental surgery than use a command line.)
- Prover9's input/output language is easily read out of the box. Most other systems rely on TPTP, which is hard to use for beginners.

Why do I still use it?

Why Prover9?

There are several good ATP systems: E, Vampire, Twee, iProver, etc. So why do mathematicians who use ATPs, still prefer PROVER9, even though until recently, it was unsupported legacy software?

Primary reasons:

- Prover9 had an OK GUI. (Remember: most software users would rather have painful dental surgery than use a command line.)
- Prover9's input/output language is easily read out of the box. Most other systems rely on TPTP, which is hard to use for beginners.

Why do I still use it?

- Inertia.

Why Prover9?

There are several good ATP systems: E, Vampire, Twee, iProver, etc. So why do mathematicians who use ATPs, still prefer PROVER9, even though until recently, it was unsupported legacy software?

Primary reasons:

- Prover9 had an OK GUI. (Remember: most software users would rather have painful dental surgery than use a command line.)
- Prover9's input/output language is easily read out of the box. Most other systems rely on TPTP, which is hard to use for beginners.

Why do I still use it?

- Inertia.
- Hints, implementing Veroff's proof sketches strategy. Other systems (E, Twee) have hints, but I've not been able to exploit them sufficiently well.

Equational logic

All general purpose ATP systems can do various things quite well, but for applications to algebra (my main interest), where they really shine is at equational reasoning. This has been the case since McCune first used EQP to solve the Robbins Problem.

This means that when formulating a problem for an ATP system, we sometimes have to think “not-so-high level” to get problems down to something manageable.

Group theory example

Consider what it means for a group G to have nilpotency class at most 3. A mathematician is trained to think of it as saying there is an upper central series

$$1 = Z_0(G) \trianglelefteq Z_1(G) \trianglelefteq Z_2(G) \trianglelefteq Z_3(G) = G$$

where $Z_{i+1}(G)/Z_i(G) = Z(G/Z_i(G))$. All very high level stuff.

Group theory example

Consider what it means for a group G to have nilpotency class at most 3. A mathematician is trained to think of it as saying there is an upper central series

$$1 = Z_0(G) \trianglelefteq Z_1(G) \trianglelefteq Z_2(G) \trianglelefteq Z_3(G) = G$$

where $Z_{i+1}(G)/Z_i(G) = Z(G/Z_i(G))$. All very high level stuff.

But that can all be expressed by one identity:

$$[x, [y, [z, u]]] = 1$$

where $[a, b] = a^{-1}b^{-1}ab$

Sample input

```
% group axioms
1 * x = x. x * 1 = x.
x' * x = 1. x * x' = 1.
(x * y) * z = x * (y * z).

% commutator
K(x,y) = x' * (y' * (x * y)).

% nilpotency class at most 3
K(x,K(y,K(z,u))) = 1.
```

Quasigroups

A *quasigroup* $(Q, \cdot)$ is a set Q with a binary operation $\cdot$ such that for each $a, b \in Q$, the equations $ax = b$ and $ya = b$ have unique solutions $x, y \in Q$.

A *loop* is a quasigroup with an identity element.

Quasigroups

A *quasigroup* $(Q, \cdot)$ is a set Q with a binary operation $\cdot$ such that for each $a, b \in Q$, the equations $ax = b$ and $ya = b$ have unique solutions $x, y \in Q$.

A *loop* is a quasigroup with an identity element.

Multiplication tables of loops = reduced Latin squares

Example:	1	2	3	4	5
	2	1	4	5	3
	3	4	5	1	2
	4	5	2	3	1
	5	3	1	2	4

Computer scientists are often more familiar with quasigroups and latin squares than mathematicians are, because they show up in cryptography, error-correcting codes, and especially, the Quasigroup Completion Problem in complexity theory.

Equational characterization

A quasigroup $(Q, \cdot, \backslash, /)$ is a set Q with three binary operations $\cdot, \backslash, /$ such that for all $x, y \in Q$:

$$x \backslash (xy) = y$$

$$x(x \backslash y) = y$$

$$(xy)/y = x$$

$$(x/y)y = x$$

So loops and quasigroups form *varieties* in the universal algebra sense of the word.

Equational characterization

A quasigroup $(Q, \cdot, \backslash, /)$ is a set Q with three binary operations $\cdot, \backslash, /$ such that for all $x, y \in Q$:

$$x \backslash (xy) = y$$

$$x(x \backslash y) = y$$

$$(xy)/y = x$$

$$(x/y)y = x$$

So loops and quasigroups form *varieties* in the universal algebra sense of the word.

The equational characterization is better suited to automated deduction. Making the division operations explicit is essentially skolemizing by hand.

Moufang loops

A *Moufang loop* is a loop satisfying any of the following equivalent identities:

$$\begin{array}{ll} x(y(xz)) = ((xy)x)z & ((zx)y)x = z(x(yx)) \\ x((yz)x) = (xy)(zx) & (x(yz))x = (xy)(zx) \end{array}$$

Moufang loops

A *Moufang loop* is a loop satisfying any of the following equivalent identities:

$$\begin{array}{ll} x(y(xz)) = ((xy)x)z & ((zx)y)x = z(x(yx)) \\ x((yz)x) = (xy)(zx) & (x(yz))x = (xy)(zx) \end{array}$$

Examples:

- groups
- the nonzero octonions
- the sphere S^7 , a subloop of the preceding example
- ... and many others

Kunen's Moufang quasigroups paper

The first paper applying automated deduction to quasigroup theory:

Kenneth Kunen, Moufang Quasigroups, *Journal of Algebra*, Volume 183, Issue 1, 1996, pp. 231–234.

Abstract: “Each of the Moufang identities in a quasigroup implies that the quasigroup is a loop.”

From the introduction: “Our investigations have been aided by the automated deduction tool, OTTER, developed by McCune. OTTER can prove theorems in full first-order logic, but it has been particularly useful in equational reasoning. . . . Its proofs are long sequences of equations, and at first sight seem a bit inscrutable. However. . . by examining these proofs. . . , one can often produce proofs which a person can easily verify by hand. . . .”

Kunen's other early quasigroup work

Ken wrote four more papers on quasigroup/loop theory, all of which used OTTER in the same way:

Quasigroups, loops, and associative laws *J. Algebra* **185** (1996), no. 1, 194–204.

Alternative loop rings *Comm. Algebra* **26** (1998), no. 2, 557–564.

G-loops and permutation groups, *J. Algebra* **220** (1999), no. 2, 694–708.

The structure of conjugacy closed loops, *Trans. Amer. Math. Soc.* **352** (2000), no. 6, 2889–2911.

Pretty good considering this was mostly a side project for him!

Diassociativity

A loop is *diassociative* if every subloop which can be generated by no more than two elements is associative.

For example, expressions involving only two variables behave like they do in groups, e.g.,

$$x^{-1}(xy) = y \quad (yx)x^{-1} = y$$

$$(xx)y = x(xy) \quad (xy)x = x(yx) \quad (yx)x = y(xx)$$

Corollary (of Moufang's Theorem)

Every Moufang loop is diassociative.

(Moufang's Theorem is much deeper: If three elements associate, they generate an associative subloop.)

Various Groups

In a loop Q , the *left* and *right translations*

$$L_x : Q \rightarrow Q; \quad yL_x = xy \qquad R_x : Q \rightarrow Q; \quad yR_x = yx$$

are permutations.

Various permutation groups act on loops:

- The *multiplication group* $\text{Mlt}(Q) = \langle L_x, R_x \mid x \in Q \rangle$
- The *inner mapping group* $\text{Inn}(Q) = \text{stabilizer of } 1 \in Q$

Bruck and Paige

Definition

A loop is *automorphic* (or an *A-loop*, for short) if every inner mapping is an automorphism.

These were introduced by Bruck and Paige in

Loops whose inner mappings are automorphisms,
Ann. of Math. (2) **63** (1956), 308–323.

Bruck and Paige

Definition

A loop is *automorphic* (or an *A-loop*, for short) if every inner mapping is an automorphism.

These were introduced by Bruck and Paige in

Loops whose inner mappings are automorphisms,
Ann. of Math. (2) **63** (1956), 308–323.

(This was the last time a loop theory paper appeared in the
Annals of Mathematics.)

Bruck and Paige

Definition

A loop is *automorphic* (or an *A-loop*, for short) if every inner mapping is an automorphism.

These were introduced by Bruck and Paige in

Loops whose inner mappings are automorphisms,
Ann. of Math. (2) **63** (1956), 308–323.

(This was the last time a loop theory paper appeared in the
Annals of Mathematics.)

B & P's (implicit) question: *Is every diassociative A-loop Moufang?*

Bruck and Paige

Definition

A loop is *automorphic* (or an *A-loop*, for short) if every inner mapping is an automorphism.

These were introduced by Bruck and Paige in

Loops whose inner mappings are automorphisms,
Ann. of Math. (2) **63** (1956), 308–323.

(This was the last time a loop theory paper appeared in the
Annals of Mathematics.)

B & P's (implicit) question: *Is every diassociative A-loop Moufang?*

The commutative case was answered affirmatively by M. Osborn (Bruck's colleague) in 1958. Osborn's proof is a *tour de force* of human equational reasoning.

Not-so-high level

$\text{Inn}(Q)$ has a set of canonical generators:

$$T_x = R_x L_x^{-1} \quad (\text{generalized conjugations})$$

$$L_{x,y} = L_x L_y L_{yx}^{-1} \quad (\text{measures of}$$

$$R_{x,y} = R_x R_y R_{xy}^{-1} \quad \text{nonassociativity})$$

Not-so-high level

$\text{Inn}(Q)$ has a set of canonical generators:

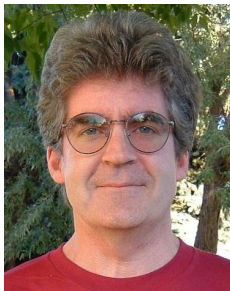
$$T_x = R_x L_x^{-1} \quad (\text{generalized conjugations})$$

$$L_{x,y} = L_x L_y L_{yx}^{-1} \quad (\text{measures of}$$

$$R_{x,y} = R_x R_y R_{xy}^{-1} \quad \text{nonassociativity})$$

Thus the high level A-loop condition $\text{Inn}(Q) \leq \text{Aut}(Q)$ can be expressed as three universally quantified equations.

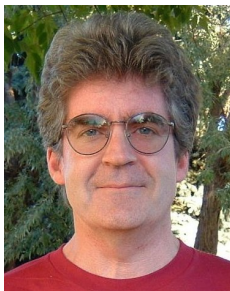
Re-enactment



J.D. Phillips



Re-enactment

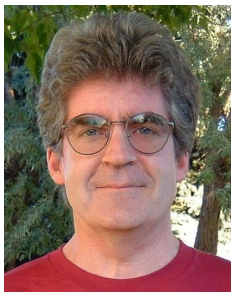


J.D. Phillips



Here's a good
problem

Re-enactment



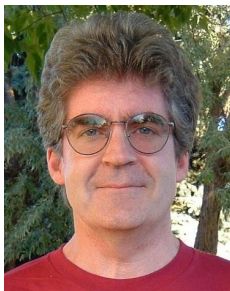
OK, let's try it.

J.D. Phillips



Here's a good
problem

Re-enactment

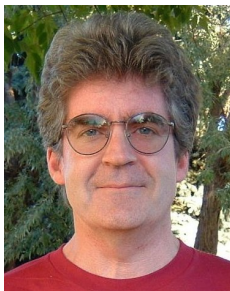


blah Moufang
loops blahblah

J.D. Phillips



Re-enactment

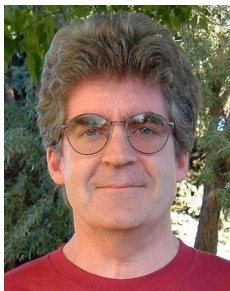


J.D. Phillips



blahblah inner
mappings blah

Re-enactment

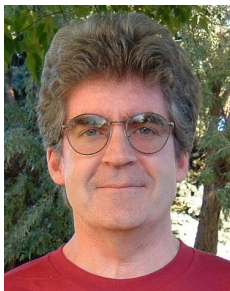


Now what?

J.D. Phillips



Re-enactment



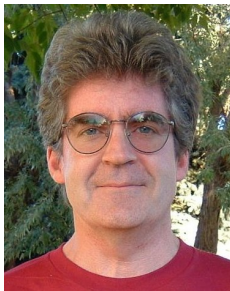
Now what?

J.D. Phillips



I dunno

Re-enactment



Hala Pflugfelder

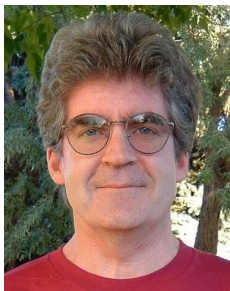


Why don't you boys
ask Ken Kunen?

J.D. Phillips



Re-enactment



Who?

Hala Pflugfelder

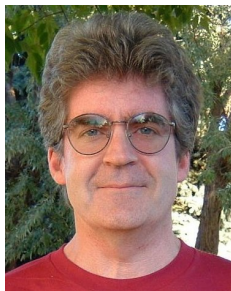


Why don't you boys
ask Ken Kunen?

J.D. Phillips



Re-enactment



Who?

Hala Pflugfelder



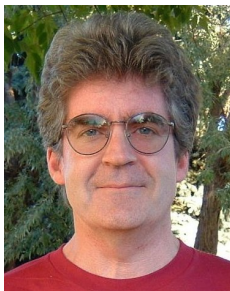
Why don't you boys
ask Ken Kunen?

J.D. Phillips



The set theorist?

Re-enactment



Hala Pflugfelder

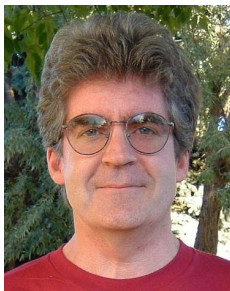


He proves theorems
about loops using
computers

J.D. Phillips



Re-enactment



Uh, OK

Hala Pflugfelder

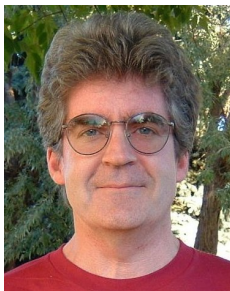


He proves theorems
about loops using
computers

J.D. Phillips



Re-enactment



Uh, OK

Hala Pflugfelder



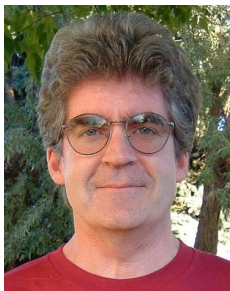
He proves theorems
about loops using
computers

J.D. Phillips



It's worth a try

Re-enactment



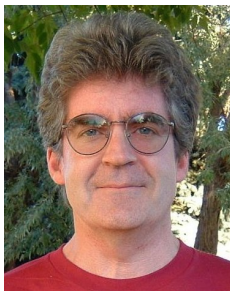
Ken Kunen



J.D. Phillips



Re-enactment



Look at this,
Prof. Kunen

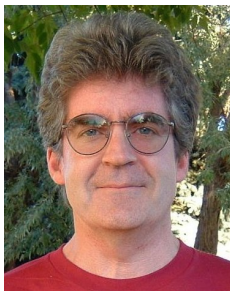
Ken Kunen



J.D. Phillips



Re-enactment



Look at this,
Prof. Kunen

Ken Kunen

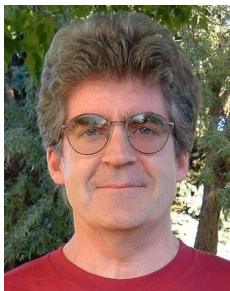


Interesting. . .

J.D. Phillips



Re-enactment

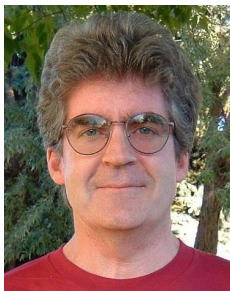


J.D. Phillips



Now what?

Re-enactment



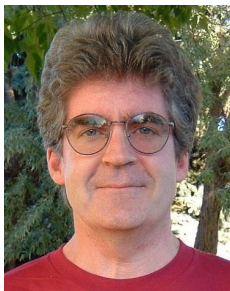
Now we wait.

J.D. Phillips



Now what?

Re-enactment

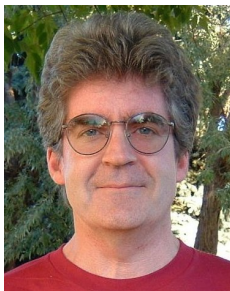


J.D. Phillips



Can we call him
“Ken”?

Re-enactment



Not yet.

J.D. Phillips



Can we call him
“Ken”?

A few days later

To: Michael Kinyon, J.D. Phillips

A few days later

To: Michael Kinyon, J.D. Phillips
From: Ken Kunen

A few days later

To: Michael Kinyon, J.D. Phillips
From: Ken Kunen
Subject: an ugly proof

A few days later

To: Michael Kinyon, J.D. Phillips

From: Ken Kunen

Subject: an ugly proof

Dear Michael and J.D.,

A few days later

To: Michael Kinyon, J.D. Phillips

From: Ken Kunen

Subject: an ugly proof

Dear Michael and J.D.,

And so began my work in this area....

The OTTER proof was about 300 steps. By today's standards, that's not very long, but it seemed daunting at the time.

PROVER9 proofs of the same result are much shorter.

Humanization

In Ken's earlier work, the proofs were much shorter, so Ken would just humanize a proof beginning-to-end. I followed his lead and over several days, we slowly inched forward.

J.D. was the smart one. He started at the goal and worked backward until he ran into facts about A-loops we already knew. Lesson learned!

An important part of humanization is reintroducing high level concepts as you go (if possible).

MK, K. Kunen, J.D. Phillips, Every diassociative A-loop is Moufang, *Proc. Amer. Math. Soc.*, volume 130, no. 3, pp 619–624.

The published version only resembles the ugly version in a few places, notably one calculation it's hard to imagine a human ever figuring out.

More papers

Since 2000, I've written ~ 42 quasigroup/loop papers which used OTTER/Prover9 in some substantial way. Many of those were with Ken or J.D. or both. Besides the ones with me, J.D. has plenty of his own.

More papers

Since 2000, I've written ~ 42 quasigroup/loop papers which used OTTER/Prover9 in some substantial way. Many of those were with Ken or J.D. or both. Besides the ones with me, J.D. has plenty of his own.

The main reason I adapted automated deduction into my toolkit:

We do what we must because we can.

– motto of Aperture Science Inc.



More on A-loops

We did eventually return to studying automorphic loops, after letting them sit idle for about 11 years. This happened because I had the Windows GUI version of PROVER9 on my laptop. I was in the backseat of my colleague's car, and his visitor suddenly asked me:

In a commutative A-loop, is it true that the product of squares is a square?

I knew how to check:

```
(A * A) * (B * B) != x * x # answer(x) .
```

Prover9 found a proof and a value of x (much to the surprise of my colleague and his visitor). That turned out to be the key technical lemma that unlocked the whole theory of commutative A-loops.

AIM: nuclei and center

The *nuclei* of a loop Q :

$$\text{Nuc}_\ell(Q) = \{a \in Q : ax \cdot y = a \cdot xy, \forall x, y \in Q\}$$

$$\text{Nuc}_m(Q) = \{a \in Q : xa \cdot y = x \cdot ay, \forall x, y \in Q\}$$

$$\text{Nuc}_r(Q) = \{a \in Q : xy \cdot a = x \cdot ya, \forall x, y \in Q\}$$

$$\text{Nuc}(Q) = \text{Nuc}_\ell(Q) \cap \text{Nuc}_m(Q) \cap \text{Nuc}_r(Q)$$

Each nucleus is a (not necessarily normal) subloop.

The *center* of a loop Q is the set of all elements that commute *and associate* with all other elements:

$$Z(Q) = \text{Nuc}(Q) \cap \{a \in Q : ax = xa, \forall x, y \in Q\}.$$

This is always a normal subloop.

AIM: commutators and associators

To measure noncommutativity and nonassociativity of a loop, we introduce commutators and associators. The convention used in this project has been:

```
% commutators and associators
```

```
K(x,y) = (y * x) \ (x * y) .
```

```
a(x,y,z) = (x * (y * z)) \ ((x * y) * z) .
```

(The AIM project has been going on for over 20 years, so I no longer remember why we used an upper case K but a lower case a.)

AIM: some more clauses

“The quotient by the center is commutative”

$$K(x, K(y, z)) = 1.$$

$$a(K(x, y), z, u) = 1.$$

$$a(x, K(y, z), u) = 1.$$

$$a(x, y, K(z, u)) = 1.$$

“The quotient by the center is a group”

$$K(x, a(y, z, u)) = 1.$$

$$a(a(x, y, z), u, w) = 1.$$

$$a(x, a(y, z, u), w) = 1.$$

$$a(x, y, a(z, u, w)) = 1.$$

AIM: Abelian Inner Mappings

```
% generators of Inn(Q)
(y * x) \ (y * (x * u)) = L(u, x, y) .
((u * x) * y) / (x * y) = R(u, x, y) .
x \ (y * x) = T(y, x) .
```

```
% AIM (Abelian Inner Mappings)
```

```
T(T(x, y), z) = T(T(x, z), y)           # label("TT")
T(L(u, x, y), z) = L(T(u, z), x, y)      # label("TL")
T(R(u, x, y), z) = R(T(u, z), x, y)      # label("TR")
L(L(u, x, y), z, w) = L(L(u, z, w), x, y) # label("LL")
L(R(u, x, y), z, w) = R(L(u, z, w), x, y) # label("LR")
R(R(u, x, y), z, w) = R(R(u, z, w), x, y) # label("RR")
```

Strong AIM Conjecture

Conjecture (Strong AIM)

Let Q be an AIM loop. Then:

- *$Q/\text{Nuc}(Q)$ is an abelian group, and*
- *$Q/Z(Q)$ is a group.*

Spoiler: No proof nor counterexample to the SAC is known

That said, the SAC has been proven in many important classes of loops, whose names would mean nothing to this audience.

AIM: the SAC's seven goals

```
K(x, a(y, z, u)) = 1 # label("Ka").  
a(K(x, y), z, u) = 1 # label("aK1").  
a(x, K(y, z), u) = 1 # label("aK2").  
a(x, y, K(z, u)) = 1 # label("aK3").  
a(a(x, y, z), u, w) = 1 # label("aa1").  
a(x, a(y, z, u), w) = 1 # label("aa2").  
a(x, y, a(z, u, w)) = 1 # label("aa3").
```

- aK1,aK2,aK3,aa1,aa2,aa3 say $Q/\text{Nuc}(Q)$ is an abelian group.
- Ka,aa1,aa2,aa3 say $Q/Z(Q)$ is a group.

Weak AIM Conjecture

Conjecture (Weak AIM)

Every AIM loop is nilpotent of class at most 3.

As the names suggest, the SAC implies the WAC.

The WAC has 32 goal clauses, some of which have a lot of variables, for example:

```
K(a(K(x,y),z,u),w) = 1 # label("nil3_aK1-C").  
a(a(a(x,y,z),u,w),v5,v6) = 1 # label("nil3_aa1-N1")
```

The ones with no more than 5 variables have been proven.

The WAC is **true**!

The proof is not a collection of PROVER9 proofs; it is rather a mix of such things along with high-level reasoning.

Recently, Josef Urban and his Gang have formalized the proof of the WAC in Megalodon. (I don't actually know what Megalodon is.)

Bob Veroff would still like to get direct ATP proofs of the 32 WAC goals. Consider this whole AIM discussion to have been an advertisement for his talk.

And now for something completely different

I remind you of my talk's subtitle:

*... How I Learned to Stop Worrying and Start Letting
Computers Prove Theorems*

Here is an example from outside the world of ATPs.

And now for something completely different

I remind you of my talk's subtitle:

*... How I Learned to Stop Worrying and Start Letting
Computers Prove Theorems*

Here is an example from outside the world of ATPs.

Theorem (M. Giudici & MK 2026)

*Let Q be a finite simple nonassociative loop. Then $\text{Mlt}(Q)$ is of
affine type or is almost simple.*

And now for something completely different

I remind you of my talk's subtitle:

... How I Learned to Stop Worrying and Start Letting Computers Prove Theorems

Here is an example from outside the world of ATPs.

Theorem (M. Giudici & MK 2026)

Let Q be a finite simple nonassociative loop. Then $\text{Mlt}(Q)$ is of affine type or is almost simple.

Proofs of three of the key lemmas were found by ChatGPT Pro and Claude Fable.

Good news: the proofs are correct, we understand them, and the citations are correct too.

Let me conclude with the following, taken from Shakespeare's *Romeo and Juliet*, Act 2, Scene 2.

Romeo has already left Juliet's house, but she is so excited, she pretends to keep talking to him.

Let me conclude with the following, taken from Shakespeare's *Romeo and Juliet*, Act 2, Scene 2.

Romeo has already left Juliet's house, but she is so excited, she pretends to keep talking to him.

Juliet: Dost thou love me? I know thou wilt say 'Ay',
And I will take thy word; yet, if thou swear'st,
Thou mayst prove false.

Let me conclude with the following, taken from Shakespeare's *Romeo and Juliet*, Act 2, Scene 2.

Romeo has already left Juliet's house, but she is so excited, she pretends to keep talking to him.

Juliet: Dost thou love me? I know thou wilt say 'Ay',
And I will take thy word; yet, if thou swear'st,
Thou mayst prove false.

As you see, Juliet is worried that Romeo will end up just like Lean.

Thank you all for listening!!!



William McCune (1953–2011)



Kenneth Kunen (1943–2020)